

Coursework 1 - Cognitive Robotics and Computer Vision

João Augusto Lopes

November 2025

1 Introduction

Robots need to see and understand their surroundings. That's the core challenge. Historically, engineers tackled this with hand-crafted geometric features. But deep learning changed everything. Now, learned representations dominate the field.

This report compares both approaches for multi-class object recognition. On one side, a classical pipeline using SIFT features and Bag of Visual Words. On the other, two deep learning architectures: a custom shallow CNN (DeepCNN) and a pretrained ResNet34. These on two very different datasets were tested. The RGB-D Object Dataset is controlled and clean. COCO is cluttered and chaotic. By tuning hyperparameters and analysing where models fail, this study explores the trade-off between simplicity and robustness in robotic vision.

2 Contextualisation of Computer Vision in Robotics

Computer vision lets robots perceive and interpret their environment. Without it, tasks like navigation and manipulation become impossible. The field started with classical, feature-based methods. These offered geometric reasoning at low computational cost.

Lowe's SIFT features became a cornerstone [1]. Their invariance to scale and rotation made them invaluable for localisation and mapping. Later, ORB features tackled speed constraints [2]. They achieved strong matching performance more efficiently, which suited embedded systems with tight latency requirements. The ORB-SLAM2 system showed how to integrate these features into full SLAM pipelines [3]. It became a standard baseline for mobile robots and drones.

Then deep learning arrived and reshaped everything. AlexNet proved that large convolutional networks could dominate image classification [4]. That sparked a shift away from hand-crafted features. ResNet pushed things further by introducing residual connections [5]. These allowed training of much deeper networks. Today, ResNet backbones appear across robotics for recognition, depth estimation, and grasp detection.

Detection and segmentation benefited too. Faster R-CNN brought efficient region proposals, enabling real-time detection for manipulation and navigation [6]. U-Net's encoder-decoder design made pixel-level understanding practical [7]. It now supports terrain classification, workspace analysis, and even surgical robotics. Deep learning also transformed manipulation research. Saxena et al. demonstrated that CNNs could learn grasp affordances directly from images [8]. That work laid the foundation for modern visuomotor grasping systems.

More recently, transformers have started reshaping the state of the art. Vision Transformers treat images as patch sequences and apply global self-attention [9]. This enables long-range reasoning that CNNs struggle with. It's particularly useful in cluttered scenes where global context matters. DETR extended transformers to object detection [10]. It eliminated hand-crafted components like anchor boxes in favour of end-to-end attention. The result is simpler, more modular systems.

The trajectory is clear. Computer vision in robotics has moved from geometric, hand-engineered features to learned representations, and now to transformer architectures. Each shift addressed growing demands for robustness, semantic understanding, and adaptability. These demands only intensify as robots leave controlled factories and enter unpredictable real-world environments.

3 Datasets

Two datasets were used: COCO and the RGB-D Object Dataset. They offer complementary challenges. Together, they enable training and evaluation across different levels of scene complexity. To keep things consistent, only nine classes common to both were retained: apple, banana, bowl, cell phone, keyboard, orange, scissors, toothbrush, and water bottle.

COCO contains complex scenes with multiple objects per image. But the goal here was isolated object recognition. So bounding boxes for the selected classes were extracted and objects cropped out. This was necessary because COCO images are cluttered. The RGB-D dataset, by contrast, provides clean single-object images. Cropping aligned the two datasets and ensured models focused on objects themselves, not background noise.

The RGB-D Object Dataset provides synchronised RGB and depth images from multiple viewpoints. For this project, only RGB images were used. Depth data, point clouds, and pose annotations were left out. Sticking to RGB allowed direct comparison with cropped COCO images and kept the dataset structure consistent.



(a) COCO Example



(b) Cropped COCO



(c) RGBD Example

Figure 1: Dataset Examples

4 CNN

4.1 Methods

4.1.1 ResNet34

The main deep learning experiments used ResNet34. This architecture solves the vanishing gradient problem through residual connections. Instead of learning complete mappings, the network learns residual functions. Figure 2 shows the structure. It starts with a convolution and pooling layer, passes through four stages of residual blocks, and ends with a fully connected classifier. Each block has two 3×3 convolutions with batch normalisation and ReLU. A shortcut connection adds the input directly to the output.

The implementation started with a ResNet34 pretrained on ImageNet. The final fully connected layer was swapped out. In its place: a dropout layer followed by a linear layer sized for the target classes. Dropout adds regularisation. The linear layer adapts the network to this specific task.

Standard preprocessing was applied. Images were resized, randomly flipped, rotated, and colour-jittered. Normalisation used ImageNet statistics. Data was split 80/20 for training and validation. Cross-entropy loss was the objective and Adam optimisation was used. Accuracy and loss were tracked each epoch.

4.1.2 DeepCNN

A custom CNN was also built from scratch. It has two convolutional blocks followed by a fully connected classifier. Each block contains two 3×3 conv layers with ReLU, then max-pooling and dropout. The output gets flattened and fed through a 512-unit dense layer with ReLU, another dropout, and a final linear layer for class logits. This design balances capacity against overfitting risk on limited data.

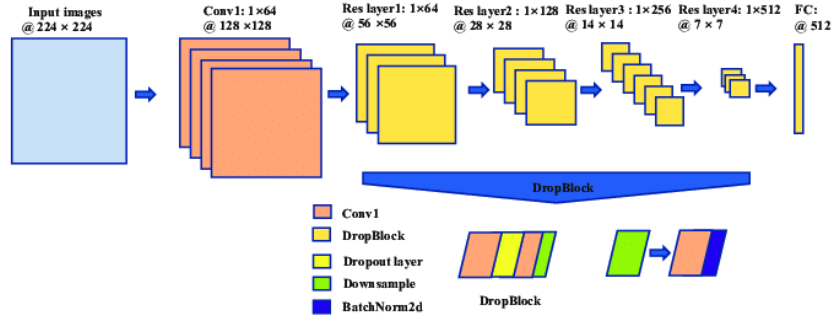


Figure 2: ResNet 34

The DeepCNN was implemented in PyTorch. Preprocessing and augmentation matched the ResNet setup. Training used separate loaders for train and validation data, cross-entropy loss, and Adam. Accuracy and loss were logged each epoch.

4.2 Hyperparameter Testing

A systematic search was run to optimise both models. Three parameters were tested: learning rate (10^{-2} , 10^{-3} , 10^{-4}), batch size (16, 32), and dropout (0.0, 0.3, 0.5). The dropout values were chosen to see how regularisation affects stability and generalisation.

Each combination trained for a limited number of epochs. Training and validation metrics were recorded throughout. The configuration with the highest validation accuracy was selected. Models were then retrained longer with those settings to confirm robust convergence.

4.3 Results

4.3.1 Hyperparameters Testing

The grid search revealed distinct optimal settings for each model-dataset pair.

On COCO, ResNet34 peaked at 82.8% validation accuracy. The winning setup: learning rate 10^{-4} , batch size 32, no dropout. Learning rate mattered most. High rates like 10^{-2} failed badly: accuracy stalled around 35-39% no matter what else we tried. Dropping to 10^{-3} helped, pushing results to about 60%. COCO’s loss landscape is clearly rough. It needs small, careful steps.

The DeepCNN struggled on COCO. Best accuracy: just 46.6%. Worse, hyperparameters barely mattered. Every configuration landed between 45-47%. The architecture itself was the bottleneck, not the training setup.

RGBD was different. Both models crushed it. ResNet34 hit 100.0% with multiple configurations. The "best" (LR 10^{-2} , batch 16, dropout 0.3) was simply the first among ties. DeepCNN reached 99.9%. Even its worst run managed 94.2%. RGBD’s classes are clearly well-separated and easy to learn.

4.3.2 With Best Hyperparameters

The best configurations were retrained for 20 epochs. This exposed important differences in learning dynamics.

ResNet34 on COCO (Figure 3a) showed training accuracy climbing to 96%. But validation plateaued at 80-81% after epoch 10. Validation loss even started rising slightly. Classic overfitting. The model memorised training data but couldn’t fully generalise.

The confusion matrix (Figure 4a) shows where errors clustered. Visually similar classes caused trouble. 18 apples were called oranges, which is understandable given their shape and colour. Bottle did well (202 correct) but attracted false positives from other elongated objects.

ResNet34 on RGBD (Figure 3b) behaved completely differently. By epoch 5, both curves exceeded 90%. By epoch 10, near-perfect. Train and validation tracked closely. No overfitting at all. The confusion matrix (Figure 4b) is a clean diagonal. Every sample correct.

But we should be cautious here. RGBD has clean backgrounds and consistent lighting. The validation set likely contains views of the same physical objects as training. The model might be recognising

instances, not learning general features.

DeepCNN on COCO (Figure 5a) underfit badly. Training accuracy reached only 56%. Validation sat around 50%. The confusion matrix (Figure 6a) shows widespread errors, many classes dumped into bottle or bowl. The network lacked capacity for COCO's complexity. On RGBD (Figure 5b), though, it performed brilliantly: 99.9% accuracy with smooth convergence. For controlled tasks, you don't need a giant network.

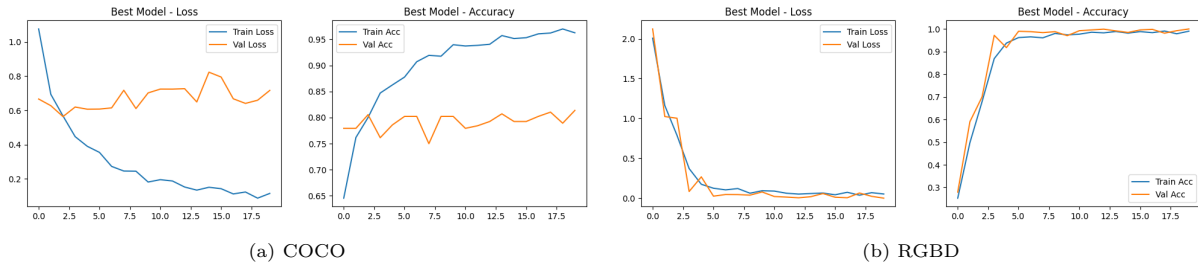


Figure 3: ResNet34 Loss and Accuracy Graph

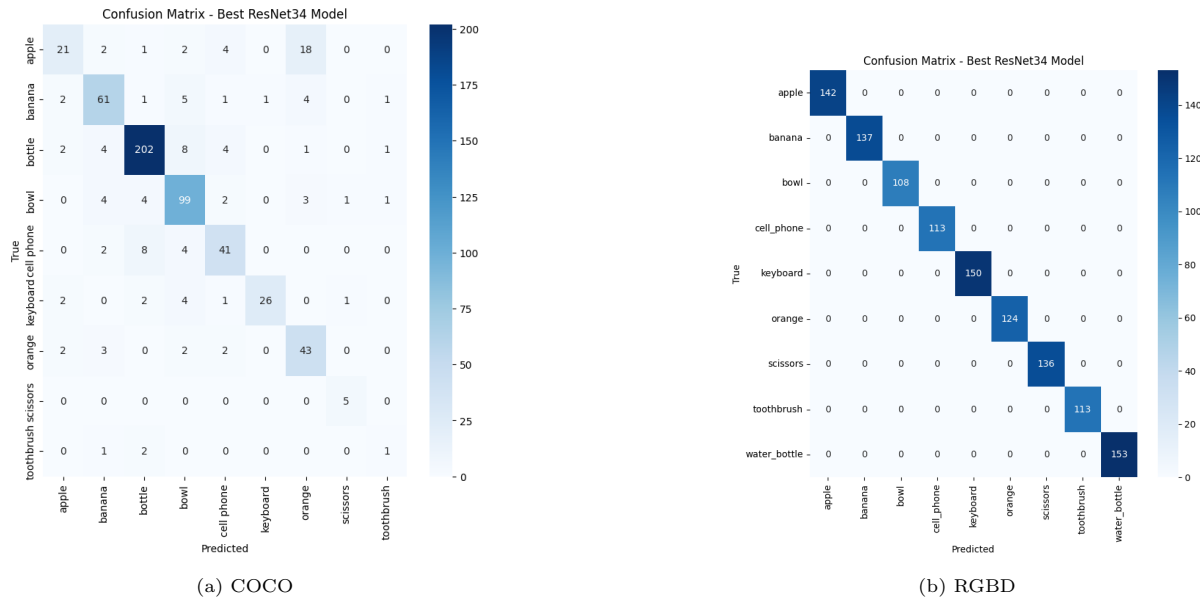


Figure 4: ResNet34 Confusion Matrix

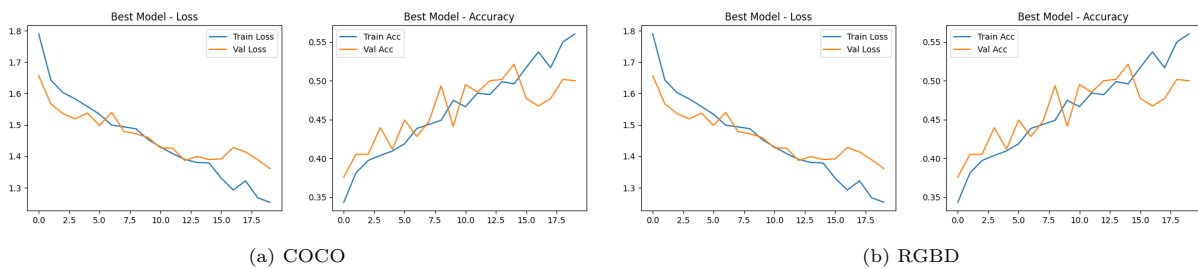
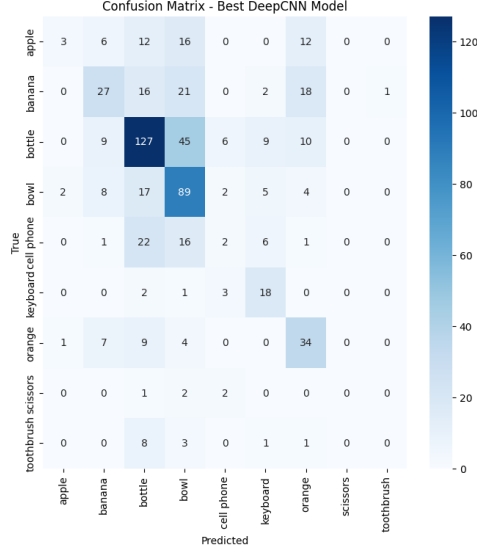
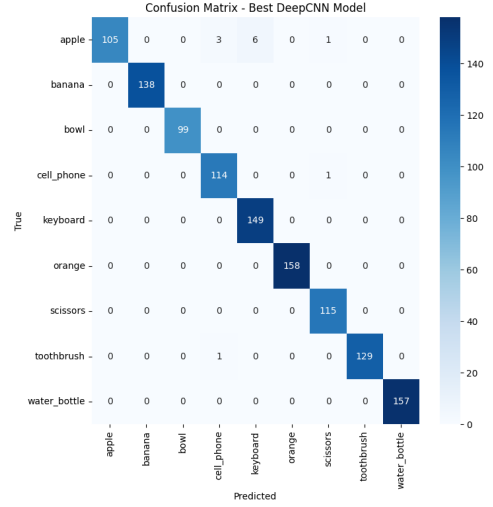


Figure 5: DeepCNN Loss and Accuracy Graph



(a) COCO



(b) RGBD

Figure 6: DeepCNN Confusion Matrices

5 Local Features

5.1 Methods

This approach uses a classical pipeline: SIFT features, Bag of Visual Words, and SVM classification. Images below 96×96 pixels were discarded since the SIFT algorithm requires sufficient spatial resolution to construct the Gaussian scale-space pyramid and detect stable keypoints across multiple octaves. Surviving images were converted to grayscale for SIFT keypoint detection and descriptor extraction. To manage computation, descriptors per image were randomly capped.

Descriptors from all images were clustered with MiniBatchKMeans to build a visual vocabulary. Each image then became a normalised histogram of visual word occurrences, a fixed-length vector for classification.

An SVM was trained on these histograms. Both linear and RBF kernels were tested. Data was split 80/20 for training and validation. Features were scaled for convergence. A sweep over SIFT contrast threshold (0.02, 0.04, 0.08), vocabulary size k (50, 100, 200), and kernel type identified the best setup.

5.2 Results

5.2.1 Hyperparameters Testing

The sweep showed stark differences between datasets, even more pronounced than with deep learning.

On RGBD, the pipeline was rock solid. Multiple configurations hit 100%. Even the worst setup (threshold 0.02, $k=50$, linear kernel) reached 98.13%. The best used threshold 0.04, $k=100$, and RBF kernel. RBF consistently beat linear by 1-2 points, suggesting some non-linear structure in the visual word distributions.

COCO was a different story. A critical problem emerged immediately: data loss. The 96×96 filter killed over 80% of images. Only 565 survived out of 3063. Many COCO objects are small or distant. Cropping them produces tiny thumbnails that fail the size check.

So we had a tiny dataset and messy images. Best accuracy: 42.73%. That required a strict SIFT threshold (0.08) to filter background noise. But tuning didn't help much. Results ranged from 28-43%. The problem wasn't parameters, it was the method itself.

5.2.2 With Best Hyperparameters

The confusion matrices reveal what's happening. On RGBD (Figure 7b), it's perfect. Every sample on the diagonal. SIFT captures local details like corners and textures. In RGBD, those stay stable and distinct. A 100-word vocabulary was enough.

On COCO (Figure 7a), the model developed a bad habit: guessing "bowl" constantly. 10 of 17 bananas? Bowl. Half the oranges? Bowl. Most scissors? Bowl.

Two reasons explain this. First, Bag of Visual Words ignores spatial layout. It doesn't know where features are, just that they exist. Smooth curves on a bowl look like smooth curves on a banana. Their histograms overlap. Second, bowl probably had more images pass the size filter, biasing the SVM. We also saw apples confused with oranges 18 times. Without colour or shape context, SIFT just sees "roundish texture". These methods work well in labs settings, but hey fall apart in the real world.

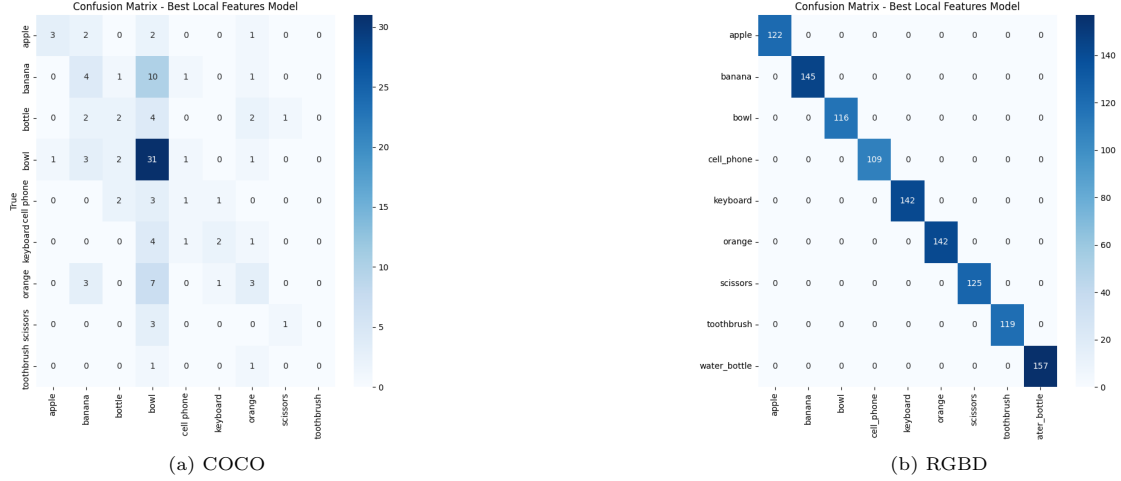


Figure 7: Local Features Confusion Matrices

6 Future Directions

The classical pipeline needs better data handling. The size filter threw out most of COCO. Upscaling strategies or Spatial Pyramid Matching could help. SPM would also preserve spatial context, potentially reducing confusion between texturally similar classes.

For deep learning, ResNet34's generalisation gap on COCO signals overfitting. Advanced augmentation like CutMix or MixUp might improve robustness. The perfect RGBD scores also raise questions. Is the dataset too easy? Cross-dataset evaluation (training on RGBD, testing on COCO) would test domain generalisation properly. Finally, explainability tools like Grad-CAM could verify whether models learn real object features or just exploit background shortcuts.

7 Conclusion - CNN vs Local Features

This project tested two approaches: deep learning (ResNet34, DeepCNN) and classical feature extraction (SIFT with Bag of Visual Words). The datasets, controlled RGB-D and chaotic COCO, revealed a clear pattern.

On RGB-D, everything worked. ResNet34: 100%. DeepCNN: 99.9%. Local Features: 100%. When lighting is consistent and objects are isolated, hand-crafted features match learned representations. Deep networks may be overkill for such tasks.

COCO told a different story. The SIFT pipeline managed only 42.73%. It couldn't handle clutter or variable resolutions. Bag of Visual Words loses spatial information, causing confusion between texturally similar classes like bowls and oranges. DeepCNN also struggled at 46.6%. It couldn't learn complex abstractions from scratch.

ResNet34 won decisively with 82.8% on COCO. Transfer learning made the difference. The pretrained backbone provided a robust feature hierarchy that neither shallow networks nor hand-crafted methods could replicate. Classical approaches still have their place for controlled tasks. But for complex robotic environments, deep learning is essential.

References

- [1] D. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, pp. 91–110, 11 2004.
- [2] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “Orb: An efficient alternative to sift or surf,” in *2011 International Conference on Computer Vision*, 2011, pp. 2564–2571.
- [3] R. Mur-Artal and J. Tardos, “Orb-slam2: an open-source slam system for monocular, stereo and rgb-d cameras,” *IEEE Transactions on Robotics*, vol. PP, 10 2016.
- [4] S. Safa aldin, N. B. Aldin, and M. Aykaç, “Enhanced image classification using edge cnn (e-cnn),” vol. 40, no. 1, p. 319–332, Feb. 2023. [Online]. Available: <https://doi.org/10.1007/s00371-023-02784-3>
- [5] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [6] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, 06 2015.
- [7] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” vol. 9351, 10 2015, pp. 234–241.
- [8] I. Lenz, H. Lee, and A. Saxena, “Deep learning for detecting robotic grasps,” *International Journal of Robotics Research*, vol. 34, 01 2013.
- [9] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” 2021.
- [10] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” in *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part I*. Berlin, Heidelberg: Springer-Verlag, 2020, p. 213–229. [Online]. Available: https://doi.org/10.1007/978-3-030-58452-8_13

A Local Features Code

```
import cv2
import os
import numpy as np
from sklearn.cluster import MiniBatchKMeans
from sklearn.svm import SVC
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

#Settings

dataset_path = "/home/laptop3/UoM/CognitiveRobotics/Coursework/coco_crops"
dataset_path = "/home/laptop3/UoM/CognitiveRobotics/RGBD"

min_width, min_height = 96, 96

# Hyperparameters to test
k_values = [50, 100, 200]
sift_thresholds = [0.04, 0.02, 0.08]
svm_kernels = ["linear", "rbf"]

max_descriptors_per_image = 1000

# Store results
final_results = []

# Extract SIFT descriptors from all images
def extract_sift_descriptors(contrastThreshold):

    sift = cv2.SIFT_create(contrastThreshold=contrastThreshold)

    all_descriptors = []
    image_descriptors = []
    image_labels = []
    used, skipped = 0, 0

    print(f"\nExtracting SIFT features (contrastThreshold={contrastThreshold})")

    for class_name in sorted(os.listdir(dataset_path)):
        class_folder = os.path.join(dataset_path, class_name)
        if not os.path.isdir(class_folder):
            continue

        for img_name in os.listdir(class_folder):
            img_path = os.path.join(class_folder, img_name)
            img = cv2.imread(img_path)
            if img is None:
```



```

        continue

    h, w = img.shape[:2]
    if w < min_width or h < min_height:
        skipped += 1
        continue

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    kp, des = sift.detectAndCompute(gray, None)

    if des is None or len(des) == 0:
        continue

    used += 1
    image_descriptors.append(des)
    image_labels.append(class_name)

    # random sample descriptors for vocabulary
    if len(des) > max_descriptors_per_image:
        idx = np.random.choice(len(des), max_descriptors_per_image, replace=False)
        sampled_des = des[idx]
    else:
        sampled_des = des

    all_descriptors.append(sampled_des)

print(f"Total images used: {used}, skipped: {skipped}")
return image_descriptors, image_labels, np.vstack(all_descriptors)

# Compute BoVW histograms for each image
def compute_bovw_histograms(kmeans, image_descriptors):
    X = []
    k = kmeans.n_clusters

    for des in image_descriptors:
        words = kmeans.predict(des)
        hist, _ = np.histogram(words, bins=np.arange(k + 1))

        hist = hist.astype(np.float32)
        hist /= (np.linalg.norm(hist) + 1e-6)

    X.append(hist)

    return np.array(X)

# Hyperparameter test sweep

best_overall_acc = 0
best_config = None
best_cm = None

```

```

for thresh in sift_thresholds:

    # SIFT descriptors
    image_descriptors, image_labels, all_descriptors = extract_sift_descriptors(thresh)

    y = np.array(image_labels)
    le = LabelEncoder()
    y_encoded = le.fit_transform(y)

    for k in k_values:

        print(f"Building vocabulary: {k}")

        kmeans = MiniBatchKMeans(
            n_clusters=k,
            batch_size=2000,
            random_state=42
        )
        kmeans.fit(all_descriptors)

        #Build BoVW features
        X = compute_bovw_histograms(kmeans, image_descriptors)
        print(f"Feature matrix shape: {X.shape}")

        # Train/test split
        X_train, X_test, y_train, y_test = train_test_split(
            X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
        )

        #Scale histograms
        scaler = StandardScaler(with_mean=False)
        X_train = scaler.fit_transform(X_train)
        X_test = scaler.transform(X_test)

        for kernel in svm_kernels:

            print(f"\nTraining SVM (kernel={kernel})...")

            if kernel == "linear":
                svm = SVC(kernel="linear", class_weight="balanced")
            else:
                svm = SVC(kernel="rbf", C=10, gamma="scale", class_weight="balanced")

            svm.fit(X_train, y_train)
            y_pred = svm.predict(X_test)

            acc = accuracy_score(y_test, y_pred)
            print(f"Accuracy: {acc*100:.2f}%")

            # store results
            final_results.append((thresh, k, kernel, acc))

```

```

# track best model for confusion matrix
if acc > best_overall_acc:
    best_overall_acc = acc
    best_config = (thresh, k, kernel)
    best_cm = confusion_matrix(y_test, y_pred)

# Show results
print("\nFINAL-RESULTS:")
for thresh, k, kernel, acc in final_results:
    print(f"SIFT_thresh={thresh} - | -k={k} - | -kernel={kernel} - | -acc={acc*100:.2f}%")

print("\nBest-configuration:")
print(f"SIFT_thresh={best_config[0]}, -k={best_config[1]}, -kernel={best_config[2]}")
print(f"Best-accuracy: -{best_overall_acc*100:.2f}%")

# Confusion Matrix for best model
plt.figure(figsize=(8,8))
sns.heatmap(best_cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.title("Confusion-Matrix -- Best-Local-Features-Model")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()

```